Direct-Actuation Bead Tracing versus RL-Tuned Pure Pursuit for Contour and Racetrack Following

Dileep Banothu
[Department]
[Institution]
[City, Country]
[email]

N. Sanjay Varma
[Department]
[Institution]
[City, Country]
[email]

***Abstract*—The accuracy of a Pure Pursuit (PP) tracker hinges on two hand-chosen quantities, the lookahead distance and the steering gain, and a recent line of work hands their online selection to a reinforcement-learning (RL) agent. This study examines the alternative of removing the geometric law entirely: does a small network that sets the acceleration of a point-mass "bead" directly follow a reference more closely than a PP controller with tuned or learned parameters? Our pipeline first turns any raster image into an ordered reference polyline with uniform arc-length spacing, using Otsu thresholding and Zhang–Suen thinning; it then simulates a damped point mass with bounded acceleration as a Gymnasium environment; finally, it trains one actor–critic network by three separately reported procedures: expert anchoring with Dataset Aggregation (DAgger), Proximal Policy Optimization (PPO) on the environment reward, and DAgger initialization followed by PPO. Training uses 11 circuits from the F1TENTH collection plus 120 synthetic contours; checkpoints are chosen on 4 further circuits and 50 contours; and a single final test is run on 8 circuits that play no part in training, selection, or baseline tuning, with five training seeds per learned controller. On these unseen circuits the bead trained by DAgger attains a root-mean-square tracking error of 1.06 cm, the same as the hand-written rule it imitates (1.07 cm). PPO fine-tuning worsens this to 1.77 cm even with small initial exploration noise, a lower learning rate and a behaviour-cloning term, and PPO from random weights reaches 5.12 cm after 3×10^6 steps. A Fixed PP whose gains are tuned on the training circuits, with optima inside the searched range, reaches 0.34 cm, and 0.31 cm at the bead's speed. Running the same PP law on the bead itself gives 0.28 cm, so the gap stems from the control law, not from the vehicle. All four pre-declared primary comparisons agree in sign on every test circuit and remain significant after Holm correction. Direct actuation appears more precise only under a naive protocol that scores a checkpoint on its own training circuits against PP with hand-set gains, which we report as a contrast. Within our training budgets and settings, neither imitation nor reinforcement learning produced a controller more accurate than a well-tuned PP law.**

***Index Terms*—Path tracking, Pure Pursuit, imitation learning, DAgger, Proximal Policy Optimization, contour following, skeletonization, F1TENTH, autonomous racing**

I. Introduction

Every autonomous vehicle needs a component that converts a planned path into control commands while staying close to that path, whether it is straight or tightly curved, slow or fast. On small-scale racing platforms like F1TENTH [1], this job is usually given to geometric controllers, and Pure Pursuit (PP) [2], [3] is the most common of them: it is simple to analyze and light enough for embedded processors running at high rates. PP aims the vehicle at a point located a distance L_d further along the path, so its behavior is set almost entirely by this lookahead and by a steering gain g. With a small L_d the vehicle reacts quickly but weaves on straight segments; with a large one it drives smoothly but runs wide in tight corners. Rules that grow L_d with speed ease this conflict without removing it.

A natural remedy is to let a learning agent choose these values. In the approach of Elgouhary and El-Wakeel [4], the PP equations are left unchanged and a PPO [5] agent picks (L_d, g) at every step from the current speed and the upcoming curvature; the controller stays interpretable, and only two numbers pass between the agent and the existing stack. A more radical remedy discards the geometric controller and lets the network output the actuation itself. Studies seldom place these two philosophies side by side under the same simulator and the same evaluation measures.

We carry out exactly that side-by-side study. Our starting point was a simulator, developed separately, in which a point mass (the "bead") traces a contour taken from any picture. We adapted the RL–PP environment, reward and training procedure of [4] to this simulator and posed a single question: given identical paths and identical scoring, is a network that drives the actuation directly more precise than a PP controller with learned parameters?

Such a comparison is easily biased toward the learned controller. If the network is trained and checkpoint-selected on the circuits on which it is scored, if the PP baselines keep hand-set gains, and if the teacher the network imitates is left out, direct actuation can look markedly more precise than it is. A second confound is the vehicle: the bead can accelerate in any direction, while the PP car must steer, so a difference between them may reflect the plant rather than the controller. We therefore keep the test circuits out of training, selection and tuning, tune every baseline on training data until its optimum lies inside the searched range, report the teacher in every comparison, give both plants the same lap rule, and also run the PP law on the bead. Under this protocol **the answer is negative**.

The building blocks are not new: the image front end uses

standard methods (Otsu thresholding, Zhang–Suen thinning), and the DAgger-trained bead only reproduces its geometric teacher. Our contribution is the controlled comparison and what it shows, namely that a well-tuned PP law is hard to beat:

- **a controlled comparison** of a direct-actuation network trained by DAgger [6], by PPO and by DAgger-initialized PPO, an adaptation of RL-tuned PP [4], and classical PP tuned on training circuits, with one simulator, one set of extracted references, one lap rule and one metric implementation, at native and at matched speed;
- **a same-plant control**: the car's PP law executed on the bead, and the bead's teacher aiming 1 to 10 waypoints ahead, which separate the effect of the control law from that of the vehicle;
- **the finding** that, on held-out circuits, the PP law is the most accurate controller on either plant, that the imitation policy only matches its teacher, and that PPO—also with the usual remedies for fine-tuning collapse and with a reward that pays for accuracy instead of time near the path—did not improve on it within our budget;
- **a naive-protocol contrast** that shows how test leakage and untuned baselines would have reversed this conclusion, together with pre-declared statistics and an explicit account of threats to validity.

II. Related Work

Geometric tracking. The two best-known geometric path followers are Pure Pursuit [2] and the Stanley method [7]; Snider [3] contrasts them in detail, and the vehicle models used to design them are covered in [8], [9]. Because PP reacts strongly to the choice of lookahead, many variants adapt L_d to speed or curvature, but each still uses a hand-chosen formula whose constants rarely carry over from one track to another. In racing, the path these controllers follow is commonly produced by minimum-curvature optimization [10], [11].

Learning-based control. Surveys document the wide use of deep RL for driving [12] and for robots in general [13]. Among on-policy methods, PPO [5] combined with Generalized Advantage Estimation (GAE) [14] has become the standard starting point, although its results are sensitive to many small implementation choices [15]. Off-policy methods such as Soft Actor-Critic (SAC) [16] use samples more efficiently at some cost in stability. Networks that map observations straight to commands are flexible, yet they are harder to interpret and to transfer, as racing comparisons have shown [17]. Using RL only to adjust a classical controller, as in [4], is a compromise between these extremes.

Imitation learning. Learning by copying a demonstrator goes back to ALVINN [18]; its weakness is that small mistakes take the learner into situations the demonstrator never showed, and errors then accumulate. DAgger [6] counters this by repeatedly asking the expert to label the states the learner actually reaches and adding them to the training set, which yields a no-regret guarantee. Because such a learner can only approach the quality of its teacher, RL fine-tuning is the usual way to go further—the route we examine in this work.

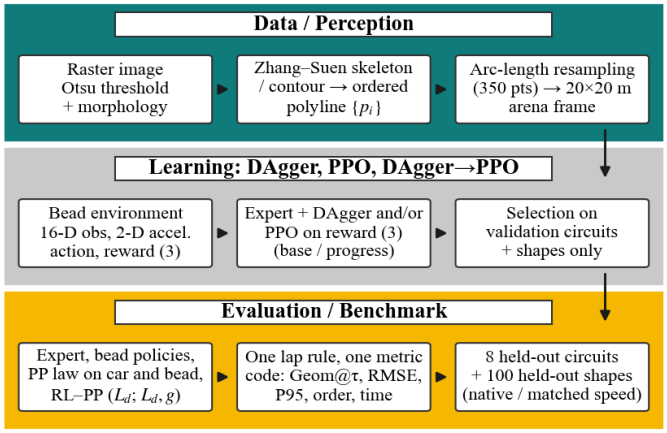


Fig. 1. System overview. Top: an input picture is turned into an ordered reference path. Middle: the bead network is trained with DAgger, with PPO, or with DAgger and then PPO, and checkpoints are picked on validation paths only. Bottom: the bead policies, the geometric expert and the Pure Pursuit family—on the car, including our adaptation of RL-tuned PP [4], and on the bead—are scored with one lap rule and the same measures on unseen circuits and shapes.

Positioning. Our bead controller lies between these directions: it is free of a PP steering equation, unlike [4], and it can lean on an inexpensive geometric teacher, unlike pure end-to-end RL. We include that teacher as a baseline in every comparison, which makes it possible to tell what the network learned apart from what the teacher and the dynamics already supply.

III. Framework and Methodology

The complete system is outlined in Fig. 1. Section III-A explains how an image becomes a reference path, Sections III-B–III-E describe the bead and how it is trained, Section III-F describes the RL–PP and classical PP baselines built on the same paths, and Section III-G the PP law run on the bead.

A. Raster-to-Reference Perception

Images larger than 1200 px on their longest side are scaled down, then converted to gray levels and smoothed with a Gaussian filter. Otsu's method [19] separates the drawing from the background; whether the drawing is darker or lighter is decided from the median brightness along the image border, so either convention works. Small holes are filled by a 3×3 closing. When the drawing covers no more than 20% of the image it is treated as a thin stroke and reduced to a one-pixel skeleton by Zhang–Suen thinning [20]; larger regions are traced along their outer contours instead. Every 8-connected piece of the skeleton is converted into an ordered list of points by walking from an end pixel and, wherever the stroke branches, continuing in the direction that turns least. The points are scaled into a 20×20 m arena with the vertical axis flipped to obtain a right-handed frame, pieces shorter than

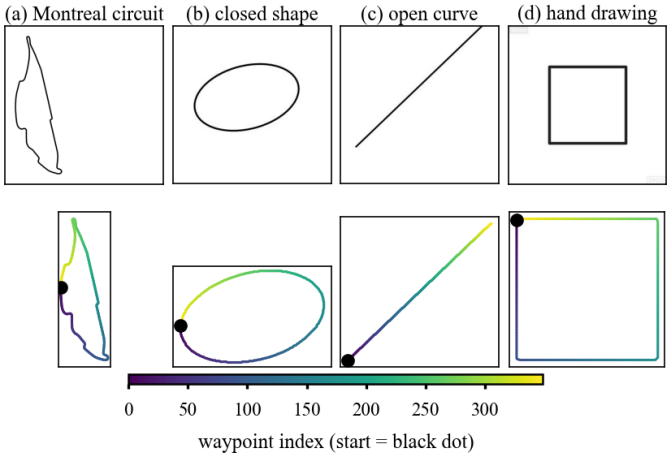


Fig. 2. Image-to-reference front end. Top: raster inputs—a rasterized test circuit, a closed shape and an open curve from the synthetic test splits, and a hand-drawn image. Bottom: the ordered references extracted from them, colored by waypoint index; the black dot marks waypoint 0.

0.1 m are dropped, and the rest are resampled at equal arc-length spacing, about 350 points in total shared in proportion to length. The junctions between separate pieces are stored as breaks and skipped in every distance computation. An image from which no path can be obtained is reported as a failure, never replaced by a default path.

Because every circuit image is rasterized from a known centerline, the extraction error can be measured exactly. Over all 23 circuits the mean distance from the source centerline to the extracted reference is 0.79–1.24 cm per circuit and the maximum is 6.53 cm. All tracking metrics are measured against the extracted reference, not against the true centerline. **Differences between controllers below about 1 cm are therefore differences in following the extracted path; they say nothing about which controller follows the real track more closely**, because the extraction error is of the same size. Rasterization also shrinks the tracks: each F1TENTH centerline is scaled to fit the 20 m arena, a linear reduction by a factor of 4.3–9.8 (median 6.5) that makes the arena circuits 39–82 m long and their corners correspondingly tighter. Fig. 2 shows the front end on four kinds of input.

B. Bead Dynamics

We model the bead as a disc of radius 0.15 m whose state is its position $\mathbf{p}_t \in \mathbb{R}^2$ and velocity $\mathbf{v}_t$. The command $\mathbf{u}_t \in [-1,1]^2$ is scaled to an acceleration of at most $a_{\max} = 10$ m/s^2 along each axis, the motion is slowed by viscous friction $\mu = 7$ s^{-1} and limited to $v_{\max} = 7$ m/s, and the state is advanced every $\Delta t = 0.016$ s:

$$\begin{aligned} \mathbf{v}_{t+1} &= (\mathbf{v}_t + a_{\max}\mathbf{u}_t\Delta t)(1-\mu\Delta t), \\ \mathbf{p}_{t+1} &= \mathbf{p}_t + \mathbf{v}_{t+1}\Delta t, \end{aligned} \tag{1}$$

where speeds above $v_{\max}$ are clipped and the bead rebounds from the arena boundary with a restitution coefficient of 0.15. Holding a full command along one axis drives (1) to the steady speed

$$v^* = a_{\max}\Delta t\,(1-\mu\Delta t)/(\mu\Delta t) \approx 1.27 \text{ m/s}, \tag{2}$$

while a full diagonal command gives at most $\sqrt{2}\,v^* \approx 1.79$ m/s; in practice the friction, rather than $v_{\max}$, limits how fast the bead can go. An episode is truncated after $\max(3500, \lceil 1.5L/(v^*\Delta t)\rceil)$ steps for a reference of length L, so that a bead moving at v^* can always finish (a fixed cap of 3500 steps would not cover the longest circuits), or after 400 consecutive steps without progress of the waypoint counter defined below. A lap is finished when that counter reaches 99% of the path. The PP vehicle uses the same counter code, lap rule, stall rule and step cap, so both plants end an episode at exactly the same point.

C. Observation and Action Spaces

The policy receives a 16-element vector $\mathbf{o}_t$ whose entries lie in $[-1,1]$, expressed relative to the bead and divided by the arena half-width $N = 10$ m. Its entries are: the vectors to the current and following waypoints (4 values); the scaled position (2) and velocity (2); the distance to the closest reference point searched within ± 20 indices (1); a flag telling whether the bead is on the path (1); the cosines of the direction of motion (2); a signed turning-angle measure of the local curvature (1); the unit vector to the waypoint five steps ahead (2); and the fraction of the path completed (1). The output is the 2-D acceleration command $\mathbf{u}_t$. Because every input is scaled to $[-1,1]$ by construction, the bead policy uses no running input normalization; the RL–PP agent of Section III-F does, because its inputs (speed in m/s, curvature in m^{-1}) have no natural scale and [4] normalizes them. The waypoint counter w_t can move forward by at most four indices per step and only to points within the coverage radius $\rho = 0.18$ m, so a controller cannot skip ahead where the path doubles back near itself; a controller that cuts a corner by more than ρ stops advancing the counter and is eventually truncated.

D. Reward Design

Writing e_t for the distance to the closest reference point, d_t for the distance to waypoint w_t, n_t for the number of waypoints passed in the current step, and $\hat{\mathbf{r}}_t$ for the unit tangent of the path, the reward at each step is

$$\begin{aligned} r_t = {} & w_a\,\mathrm{clip}\big((d_{t-1}-d_t)/\rho, -1, 1\big) + w_w n_t \\ & + w_e \exp[-(e_t/\sigma)^2] \\ & + w_\tau\,\mathrm{clip}(\mathbf{v}_t\cdot\hat{\mathbf{r}}_t/v_{\max}, -1, 1) - w_\ell \min(e_t/\rho_s, 3) \\ & - w_o\mathbf{1}[e_t > \rho] - w_b\mathbf{1}_{\text{back}} - w_c\mathbf{1}_{\text{wall}} \\ & - w_j \min(\|\mathbf{u}_t - \mathbf{u}_{t-1}\|, 1) - w_s, \end{aligned} \tag{3}$$

in which $\mathbf{1}_{\text{back}}$ flags a step backward along the path, $\mathbf{1}_{\text{wall}}$ flags contact with the boundary, $\sigma = 0.08$ m is the width of the Gaussian bonus for staying centered, and $\rho_s = 0.10$ m is the tight tolerance. At the end of an episode the agent gains $+50$ if the waypoint counter reached 99% of the path, $+15$ if the episode was cut off after passing 80%, and -5 in every other

TABLE I
REWARD WEIGHTS OF THE BEAD ENVIRONMENT

Term	Symbol	Weight	Term	Symbol	Weight
Approach	w_a	0.10	Off-curve	w_o	0.10
Waypoint	w_w	0.40	Backtrack	w_b	0.05
Centering	w_e	0.25	Wall contact	w_c	0.50
Tangency	w_τ	0.15	Action jerk	w_j	0.005
Tracking error	w_ℓ	0.05	Step cost	w_s	0.001

case. The weights, which were set before the data split and not tuned afterwards, appear in Table I. The reward is the training signal of the PPO variants (Section III-E); the DAgger variant does not use it.

Most positive terms of (3) are paid at every step, so a slower traversal collects more of them. A validation diagnostic shows that this matters: adding Gaussian action noise of standard deviation 0.61 to the expert triples its RMSE (0.91 to 2.92 cm) and lengthens its lap (48 to 61 s), yet raises its return from 746 to 764. We therefore also train with a *progress* variant in which the centering bonus is paid per waypoint passed, $2.0\, n_t \exp[-(e_t/\sigma)^2]$ instead of $w_e \exp[-(e_t/\sigma)^2]$; the factor 2.0 keeps the per-lap budget of the term at the expert's pace of about eight steps per waypoint, and all other terms are unchanged. Under this variant the same noisy expert earns less than the noise-free one (725 versus 836).

E. Policy Architecture and Learning

We use the actor–critic of Stable-Baselines3 [21], giving the actor and the critic their own two-layer networks of 256 tanh units and using a diagonal Gaussian over actions whose spread does not depend on the state. At test time the mean action is applied, with no sampling. The hand-written teacher simply steers toward the waypoint after the current one,

$$\mathbf{u}_t^* = (\mathbf{p}[w_t + 1] - \mathbf{p}_t)/\|\mathbf{p}[w_t + 1] - \mathbf{p}_t\|. \tag{4}$$

The same network is trained in three ways, which we name by the procedure that actually updates it.

Bead-DAgger first fits the actor to one teacher-driven episode on each training path and then improves it with DAgger [6]: the network drives, the teacher supplies the correct action for every state reached, and the growing dataset is fitted with a squared-error loss on the actor branch (Algorithm 1). Neither the critic nor the reward plays any part.

Bead-PPO optimizes the network from a random initialization with the clipped surrogate objective of PPO [5],

$$\mathcal{L}(\theta) = \mathbb{E}_t\big[\min(\varrho_t \hat{A}_t, \text{clip}(\varrho_t, 1 \pm \epsilon)\hat{A}_t)\big] - c_v \mathcal{L}^V + c_e \mathcal{H}, \tag{5}$$

where ϱ_t is the probability ratio between the new and the rollout policy, $\hat{A}_t$ is the GAE [14] advantage of the reward (3), $\mathcal{L}^V$ the value loss and $\mathcal{H}$ the policy entropy. Time-limit truncations are bootstrapped with the critic. The hyperparameters (Table II) were fixed in advance and not tuned, except the initial log standard deviation $\log \sigma_0$, which was chosen from $\{-1, -0.5, 0\}$ by a pilot with training seed 100 that was scored on validation references only and is not part of the results: after 7.5×10^5 steps its validation scores were 0.750, 0.851 and 0.811 (circuit RMSE 6.69, 7.12 and 7.82 cm), so $\log \sigma_0 = -0.5$ was used. Each run lasts 3×10^6 steps.

Algorithm 1 Expert anchoring and DAgger refinement (Bead-DAgger)

Input: training references $\mathcal{T}$, validation references $\mathcal{V}$, expert π^* (4), network π_θ, mixing schedule $\beta = (0.4, 0.2, 0.1, 0.05, 0, 0, 0, 0)$
$\mathcal{D} \leftarrow$ one expert episode per $\tau \in \mathcal{T}$; fit actor on $\mathcal{D}$ (12 epochs, lr 2×10^{-4})
for round $k = 1, \ldots, 8$ **do**
  for each $\tau \in \mathcal{T}$, roll out one episode executing π^* w.p. β_k, else π_θ; label every state with $\pi^*(o)$
  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(o, \pi^*(o))\}$; refit actor (8 epochs, lr 1.5×10^{-4} for $k \le 3$, else 7×10^{-5})
  evaluate the validation score on $\mathcal{V}$; keep the checkpoint if it improves
end for
return best validation checkpoint

Bead-DAgger→PPO copies the actor of the Bead-DAgger checkpoint of the same seed, trains the critic for 5×10^4 steps with the actor frozen, and then continues with (5) for 2×10^6 steps. Because a good initial policy is easily scrambled by exploration noise and large updates, fine-tuning starts with $\log \sigma_0 = -2$ ($\sigma_0 \approx 0.14$), uses the learning rate 10^{-4}, and adds a behaviour-cloning term that keeps the actor mean $\boldsymbol{\mu}_\theta$ close to the expert,

$$\mathcal{L}_{\text{BC}}(\theta) = \lambda\, \mathbb{E}_t\big[\|\boldsymbol{\mu}_\theta(\mathbf{o}_t) - \mathbf{u}_t^*\|^2\big], \quad \lambda = 1, \tag{6}$$

which is subtracted from (5) on every minibatch. The expert action is recovered exactly from the observation, whose entries 3–4 are the scaled vector to waypoint $w_t + 1$. The weight λ was set in advance and not tuned.

Both PPO procedures are run with the base reward (3) and with its progress variant, which gives four PPO-trained bead controllers. All variants sample training references from the training split, with circuits making up about half of all episodes, and keep the checkpoint with the best validation score: the mean Geom@0.10, as a fraction, minus the mean RMSE in meters, averaged with equal weight over validation circuits and validation shapes. In these units 1 cm of RMSE weighs the same as one percentage point of coverage. RMSE alone measures only how far the vehicle strays and does not notice reference sections that are skipped; Geom@0.10 penalizes skipped sections. Selecting by RMSE alone would have chosen the same checkpoint or configuration in 32 of the 45 runs and grids. In the others the selected validation RMSE differs by a median of 0.04 cm and by at most 0.48 cm. No test reference is used before the final evaluation.

F. Adaptation of RL-Tuned Pure Pursuit

To give both controllers the same paths and the same scoring, we adapted the method of [4], an arXiv preprint that has not been peer reviewed, to our arena. Because the arena,

TABLE II
LEARNING HYPERPARAMETERS

Quantity	Bead (PPO variants)	RL–PP (adaptation of [4])
Policy / value MLP	$[256, 256]$ tanh	SB3 default $[64, 64]$
Rollout n_{steps} / batch	2048×4 envs / 128	2048 / 256
Epochs per update	10	5
γ / GAE λ	0.99 / 0.95	0.99 / 0.98
Clip ϵ / target KL	0.2 / –	0.2 / 0.015
Entropy / value coeff.	0.001 / 0.5	0.02 / 0.6
Grad-norm clip	0.5	0.7
Learning rate	3×10^{-4} (scratch), 10^{-4} (fine-tune)	2.4×10^{-4}, linear decay
Initial $\log \sigma$	-0.5 (scratch, pilot), -2 (fine-tune)	0 (default)
Behaviour-cloning weight λ	– (scratch), 1 (fine-tune)	–
Normalization	– (inputs already in $[-1, 1]$)	VecNormalize (obs, return)‡
Environment steps	3×10^6 (scratch), 2×10^6 (fine-tune)	10^5 per run
Training seeds	0–4	0–4

‡Statistics saved with each checkpoint and applied at evaluation.

TABLE III
RL-TUNED PP: ORIGINAL [4] AND OUR ADAPTATION

Aspect	[4]	This work
Track scale	F1TENTH maps	maps shrunk 4.3–9.8× into 20×20 m
L_d range [m]	$[0.35, 4.0]$	$[0.05, 1.20]$
g range	$[0.45, 1.15]$	$[0.60, 1.40]$
Teacher L^*	$0.50 + 0.28v - 3.5\kappa_{\max}$	$0.65 + 0.04v - 0.18\kappa_{\max}$, in $[0.26, 0.85]$
Teacher g^*	linear, decreasing in v	$0.96 + 0.14\kappa_{\max}$, increasing in $\kappa_{\max}$
Observations	$v, \kappa_{0,1,2}, \Delta\kappa$ (5)	same + lateral error, sin / cos heading error (8)
Reward terms	speed, teacher error, jerk, $\lvert\kappa\rvert$, L_d–κ product, pre-bend bonus, collision, slow, progress	speed, teacher error (L_d and g), jerk, one bend term for L_d, collision, slow, progress + corridor bonus, quadratic error penalty, alignment bonus; other weights (Sec. III-F)
Reward clip	$[-30, 100]$	$[-30, 100]$
Speed profile	raceline, up to 15.6 m/s	friction-limited, 1.8–5.0 m/s (or v_{match})
Steering limit	PP command clipped to ± 0.35	$\lvert\delta\rvert \le 1.10$ rad
Training data	Hockenheim	11 circuits + 120 contours (training split)
PPO budget	up to 1.2×10^6 steps, Optuna	10^5 steps, no tuning
Rollout length	4096	2048
Evaluation	simulation + 1:10 car	simulation only

the speeds and the training data differ, this is an adaptation rather than a re-implementation; Table III lists every change.

The car is simulated with a kinematic bicycle model [8] of wheelbase $W = 0.33$ m. PP takes the sideways offset y' of the lookahead point in the car's own frame, converts it into a desired curvature, and turns that curvature into a steering angle:

$$\gamma = g \cdot 2y'/L_d^2, \qquad \dot{\theta} = (v/W)\tan\delta, \\ \delta = \text{clip}(\arctan(W\gamma), \pm 1.10 \text{ rad}), \tag{7}$$

As in [4], the agent sees the speed, the curvature magnitudes $\kappa_{0,1,2}$ at 0, 5 and 12 waypoints ahead, and their difference $\Delta\kappa = \kappa_1 - \kappa_0$; we add the signed sideways error and the sine and cosine of the heading error, for eight inputs in total. Its two outputs are mapped to $L_d \in [0.05, 1.20]$ m and $g \in [0.60, 1.40]$ and then low-pass filtered as in [4], with $\beta_L = \beta_g = 0.2$:

$$\tilde{L}_{t+1} = \beta_L L_{t+1} + (1 - \beta_L)\tilde{L}_t, \\ \tilde{g}_{t+1} = \beta_g g_{t+1} + (1 - \beta_g)\tilde{g}_t. \tag{8}$$

Our arena is several times smaller than a real F1TENTH track, so the original parameter ranges and teacher constants had to be scaled down. The teacher's targets are $L^* = \text{clip}(0.65 - 0.18\kappa_{\max} + 0.04v,\ 0.26,\ 0.85)$ and $g^* = \text{clip}(0.96 + 0.14\kappa_{\max},\ 0.60,\ 1.40)$. Speed follows a friction-limited target $v_{\text{ref}} = \text{clip}(4.2/\sqrt{1 + 2.5|\kappa|}, 1.8, 5.0)$ m/s, smoothed by a backward braking pass (3.5 m/s^2) and a forward acceleration pass (2.5 m/s^2), and the car tracks it with $a = 3(v_{\text{ref}} - v)$. The reward follows the structure of [4] with our own weights: $-2|L_d - L^*| - |g - g^*|$ for distance from the teacher, $-0.25|\Delta L_d| - 0.10|\Delta g|$ for abrupt parameter changes, a bend term ($+2.2$ if $L_d \le 0.40$ m when $\kappa_{\max} > 0.65$ m^{-1}, else $-4(L_d - 0.40)\kappa_{\max}$), $+1.6$ per waypoint passed, $+v/v_{\max}$, -10 on wall contact and -1 below 1 m/s. It drops the separate $|\kappa|$ term of [4] and adds three terms: a Gaussian corridor bonus $3.5\exp[-\frac{1}{2}(e/0.035)^2]$, a quadratic penalty for errors above 0.07 m, and an alignment bonus $0.8\cos e_\theta$. The sum is clipped to $[-30, 100]$ as in [4]. The joint and the L_d-only RL–PP agents are trained independently, so the ablation of Section V-H removes the gain output during training and not only at inference.

Two classical controllers complete the set: Fixed PP with constant (L_d, g) and Adaptive PP with $L_d = \text{clip}(a + bv, 0.05, 1.20)$, $g = 1.05$. Both were grid-searched on the training references with the same selection score as the learned controllers, and only settings that complete every tuning reference were eligible. The Fixed PP grid has 23 lookaheads, $L_d \in \{0.05, 0.075, \ldots, 0.20\} \cup \{0.25, 0.30, \ldots, 1.00\}$ m, times 9 gains, $g \in \{0.6, 0.7, \ldots, 1.4\}$. The Adaptive PP grid has $a \in \{0, 0.025, \ldots, 0.50\}$ and $b \in \{0, 0.025, 0.05, 0.10, \ldots, 0.25\}$ s. Fixed PP selects $(L_d, g) = (0.125 \text{ m}, 0.9)$ at native speed and $(0.125 \text{ m}, 1.0)$ at v_{match}. Adaptive PP selects $(a, b) = (0.05, 0.025)$ and $(0.05, 0.05)$, a lookahead that still grows with speed. Every selected value lies inside its grid: on the training circuits the Fixed PP error rises on both sides of the optimum (0.37 cm at 0.10 m, 0.24 cm at 0.125 m, 0.26 cm at 0.15 m, best gain at each L_d, native speed), and at $L_d \le 0.075$ m some tuning laps are not completed. The Adaptive PP schedule is nearly constant over the speeds that occur: on the tuning references, the best setting with $b = 0$ (a constant 0.125 m) scores within 0.001 cm RMSE of the selected one (0.509 versus 0.510 cm), so Adaptive PP is in effect a Fixed PP. Untuned, hand-set values—$(L_d, g) = (0.55 \text{ m}, 1.05)$ and $(a, b) = (0.35, 0.25)$—are reported alongside as part of the naive protocol (Section IV-B).

The kinematic bicycle model has no tyre forces, so nothing limits the lateral acceleration $v\dot{\theta} = v^2 \tan\delta/W$. At native speed, tuned Fixed PP demands more than 10 m/s^2, about what tyre friction allows, during 12.4% of its steps on the test circuits (99th percentile 29.3 m/s^2), because the speed profile drives even the shrunken corners at no less than 1.8 m/s. A real car would slip there, so native-speed car results are optimistic. At v_{match} the largest value is 9.4 m/s^2, within that limit.

G. Pure Pursuit on the Bead

To separate the control law from the vehicle, we also run the car's Fixed PP law on the bead (*Bead-PP*). The nearest-point window and the lookahead search are those of the car. The heading is the direction of the bead's velocity, and the PP curvature $\gamma = g \cdot 2y'/L_d^2$ of (7) is realised as a lateral acceleration $v^2\gamma$ normal to the velocity. A longitudinal acceleration, obtained by inverting one step of (1), holds the speed at v_{match}. When the per-axis bound $a_{\max}$ is reached, the longitudinal part is reduced first, so steering has priority. Bead-PP is tuned on the same references and the same (L_d, g)

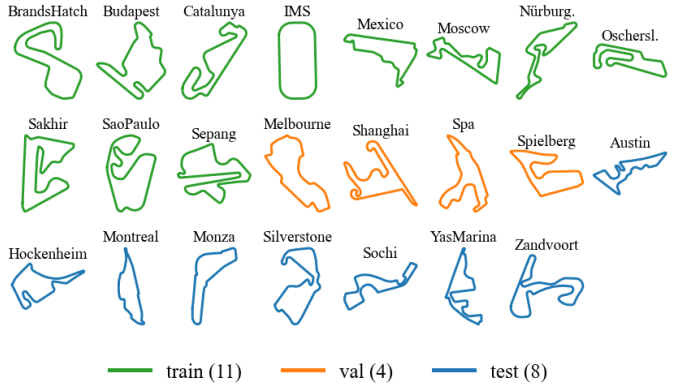


Fig. 3. Extracted references of the 23 F1TENTH circuits, colored by split. The test split contains Hockenheim, Montreal and Yas Marina, the training circuits of the naive-protocol bead, and five further circuits; no test circuit is used for training, checkpoint selection, or baseline tuning.

grid as the car, and selects $(0.125\ \text{m}, 1.2)$, also inside the grid. A second control varies the expert itself: (4) aims at waypoint $w_t + k$ instead of $w_t + 1$, for $k = 1, \ldots, 10$, either as defined, with w_t the waypoint counter, or with w_t replaced by the nearest reference point tracked as in PP. The second form keeps the target from locking when the counter stops advancing. The best k of each form is selected on the tuning references, and every k is reported on the test circuits.

IV. Experimental Setup

A. References and Datasets

We rasterized all 23 circuits of the public F1TENTH racetrack collection[1] on a 1200-px canvas with a 7-px stroke and passed them through the front end of Section III-A. Hockenheim, Montreal, and Yas Marina, on which the naive-protocol bead (Section IV-B) was trained, were assigned to the test split; the remaining 20 were split by a seeded permutation, fixed before any training, into 5 further test, 4 validation, and 11 training circuits (Fig. 3). No circuit appears in more than one split. Training additionally uses 60 closed shapes and 60 open curves from the training splits of two synthetic datasets of 1000 images each (polygons, ellipses, and rounded quadrilaterals; arcs, lines, and parabolas); 25 of each validation split are used for selection and the first 50 of each test split for testing. Synthetic splits are i.i.d. draws from the same generator, so they test in-distribution generalization; the circuits test transfer to unseen layouts. In total 493 references were passed through the front end—23 circuits, 120 training, 50 validation and all 300 test-split synthetic contours—and all were extracted without failure; of the 300 synthetic test references, the first 50 per set (100) were evaluated.

B. Naive-Protocol Baselines

To measure how much the evaluation protocol alone changes the conclusion, we also score controllers obtained under a naive protocol of the kind common in single-track studies. The naive bead uses the same network, but its actor was fitted by regression onto expert actions on Hockenheim, Montreal and Yas Marina, and its checkpoint was selected on those same circuits; scoring it on them measures test leakage directly. The naive RL–PP agent was trained on a single hand-drawn contour; its observation-normalization statistics were not stored, so it runs on raw observations. Fixed and Adaptive PP with the hand-set values of Section III-F complete the set. These rows are reported for contrast only and enter none of the pre-declared comparisons.

C. Metrics

For a reference $P = (\mathbf{p}_0, \ldots, \mathbf{p}_{N-1})$ and a driven trajectory $X = (\mathbf{x}_0, \ldots, \mathbf{x}_T)$, geometric coverage with tolerance τ is the share of reference points lying within τ of the trajectory polyline:

$$\text{Geom@}\tau = \frac{1}{N} \sum_{\mathbf{p} \in P} \mathbf{1}[\text{dist}(\mathbf{p}, X) \le \tau]. \tag{9}$$

In the other direction, the distance $E = \text{dist}(\mathbf{x}, P)$ of each trajectory sample from the reference gives the root-mean-square tracking error (RMSE), its 95th percentile (P95) and its maximum. Geom@τ does not care about the order of visits. Ordered coverage K/N counts the largest set of reference points $i_1 < \cdots < i_K$ that are each visited, $\|\mathbf{x}_{t_k} - \mathbf{p}_{i_k}\| \le \rho$, at non-decreasing times $t_1 \le \cdots \le t_K$; dynamic programming evaluates it exactly, and it punishes skipped or reversed sections while still crediting the remainder of a lap after a single excursion. We count a run as completed when its ordered coverage is 0.95 or more. A single piece of code computes every measure from the saved trajectories of all controllers, so the bead and the car are scored by identical rules. Lap time equals the number of steps times Δt.

D. Protocol

Every controller was executed for five episodes per test circuit with seeds $\{50000{+}137k\}$, $k = 0, \ldots, 4$, which perturb the initial position by up to ± 5 cm, and for one episode per test shape. Policies were run deterministically. Each learned controller was trained with five seeds (0–4); Table IV reports the mean over seeds with the range across seeds in brackets. Track A runs each controller at its native pace; Track B caps the PP reference speed at $v_{\text{match}} = 1.256$ m/s, the mean speed of the expert on the training circuits, with Fixed and Adaptive PP re-tuned and RL–PP retrained at that speed; Bead-PP runs at v_{match} as well.

Statistics. All comparisons are paired over the 8 test circuits, the value per circuit being the mean over training seeds and evaluation episodes. Our main evidence is the mean paired difference with a 95% bootstrap interval (10 000 resamples of circuits, and of training seeds within learned controllers). Hypothesis tests have little power with $n = 8$: the smallest attainable two-sided p of an exact Wilcoxon signed-rank test is 0.0078, reached only when all eight circuits agree in sign, so a large family of corrected tests cannot reject whatever the data. We therefore test only four primary comparisons,

[1] https://github.com/f1tenth/f1tenth_racetracks, commit b95c4ef.

each on RMSE and each answering one question of the study: (P1) Bead-DAgger versus tuned Fixed PP at v_{match}, i.e. direct actuation versus tuned PP at equal speed; (P2) Bead-DAgger→PPO versus Bead-DAgger, i.e. whether reward-based fine-tuning improves imitation; (P3) RL–PP joint versus tuned Fixed PP, i.e. learned versus constant PP parameters; and (P4) Bead-PP versus the expert, i.e. the control law on a fixed plant. These four were declared before the final evaluation and are corrected with Holm's method; a test can then reject only if all eight circuits agree. Every other comparison is reported with its bootstrap interval and an unadjusted p as exploratory. Episodes are not pooled into the tests, because the five episodes on a circuit differ only by a ± 5 cm start perturbation and are not independent replicates.

Table IV lists 18 controllers; counting each training seed separately they form 50 evaluated instances: 10 controllers with a single instance (the expert, Bead-PP, the six classical PP configurations, and the two naive-protocol checkpoints) and 8 learned controllers with five seeds each (Bead-DAgger, the four PPO-trained bead variants, RL–PP joint, RL–PP L_d only, RL–PP joint at v_{match}). Each instance ran $8 \times 5 = 40$ circuit episodes and 100 contour episodes, giving $50 \times 140 = 7000$ test episodes, executed on a CPU-only workstation using PyTorch 2.11, Stable-Baselines3 2.8 [21], and Gymnasium 1.2 [22]. For verification, every metric was recomputed from the stored trajectories, RMSE and Geom@0.10 additionally with an independent implementation, every deterministic controller and training seed 0 of every learned controller (18 instances) were re-executed from their saved checkpoints with bit-identical trajectories, and all tables were regenerated from the raw records; the scripts are part of the released code.

V. Results and Discussion

This section starts with accuracy on the unseen circuits and with the same-plant controls (Section V-B), then shows what the naive protocol would have reported (Section V-C), what PPO does to the bead (Section V-D), the balance between precision and speed (Section V-E) and the synthetic contours (Section V-F). It continues with the schedule learned by RL–PP (Sections V-G–V-H) and the statistics, and ends with the threats to validity.

A. Circuit-Level Tracking Accuracy

Table IV and Fig. 7 consolidate the held-out circuit results. Among the learned bead controllers, **Bead-DAgger matches its geometric expert** (RMSE 1.06 vs. 1.07 cm), with almost no spread across its five seeds. All four PPO-trained variants are worse: Bead-DAgger→PPO reaches 1.77 cm (2.14 cm with the progress reward), and Bead-PPO from scratch 5.12 cm (6.49 cm), covering 89.8% (83.1%) of the reference within 0.10 m and completing 98% (74%) of the test runs.

The PP family is more accurate. Tuned Fixed PP reaches 0.34 cm and tuned Adaptive PP 0.42 cm, both below the expert and every learned bead policy, while traversing a lap in 18.5 s instead of 42.5 s. RL–PP is less accurate than tuned constant PP (1.83 cm joint, 1.57 cm L_d only; Section V-G explains why). With the shared lap rule, every controller that completes its laps ends them at the same waypoint, so Geom@0.10 no longer differs between the plants for reasons of episode termination (99.2% for the expert, Bead-PP and tuned Fixed PP). Adaptive PP with hand-set gains completes no test lap: its lookahead of 0.8–1.6 m at the driven speeds of 1.8–5 m/s cuts corners by more than ρ, the waypoint counter stops advancing, and the run is truncated.

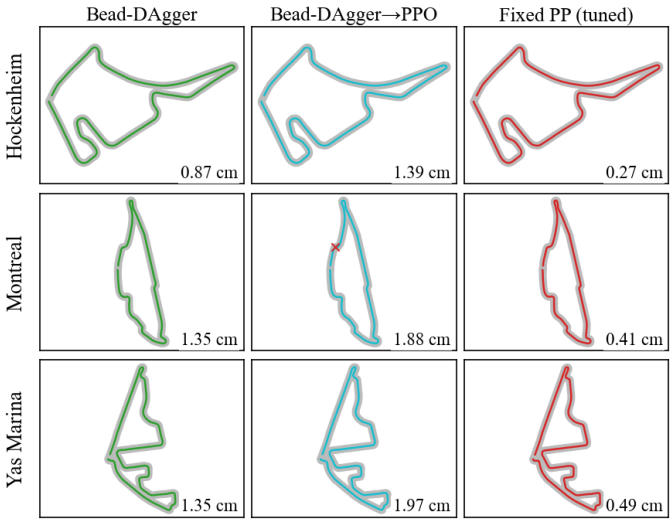


Fig. 4. Full-lap trajectories on the held-out Hockenheim, Montreal and Yas Marina circuits (first episode, training seed 0), over the reference (grey). Red crosses: reference points left farther than 0.10 m, excluding the start/finish closure. Inset: tracking RMSE of the lap.

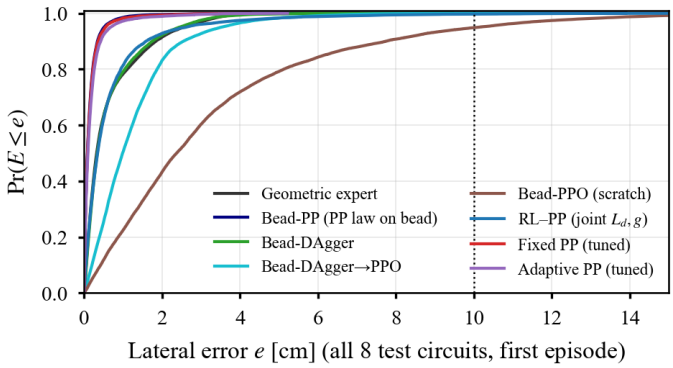


Fig. 5. Cumulative share of trajectory samples below a given distance from the reference, pooled over the eight test circuits (first episode, training seed 0 where applicable). Dotted line: 0.10 m.

Fig. 4 shows complete laps on Hockenheim, Montreal and Yas Marina, and Fig. 5 the distribution of the lateral error over all test circuits: the PP law keeps 99.8% (Bead-PP), 99.8% (Fixed PP) and 99.6% (Adaptive PP) of the trajectory samples within 3 cm of the reference, against 96.9% for the expert, 97.5% for Bead-DAgger, 96.0% for RL–PP, 92.7% for Bead-DAgger→PPO and 60.4% for Bead-PPO.

Fig. 8 shows where the bead policies fail. At the Montreal corners where Bead-DAgger→PPO deviates most, it departs slightly from the path that the expert, Bead-DAgger, tuned

TABLE IV

TRACKING PERFORMANCE ON THE EIGHT HELD-OUT TEST CIRCUITS (MEAN OVER CIRCUITS, FIVE EPISODES EACH; LEARNED CONTROLLERS: MEAN OVER FIVE TRAINING SEEDS WITH THE RANGE ACROSS SEEDS IN BRACKETS; BEST ACCURACY PER COLUMN IN BOLD; TIME AND SPEED NOT RANKED)

Controller	Geom@0.05 [%]↑	Geom@0.10 [%]↑	RMSE [cm]↓	P95 [cm]↓	Max [cm]↓	Ordered [%]↑	Completed [%]	Time [s]	Speed [m/s]
Bead plant (holonomic point mass, damping-limited)									
Geometric expert (4)	98.6	**99.2**	1.07	2.55	5.02	**99.1**	100	42.5	1.25
Bead-PP (PP law on bead)§	**99.0**	**99.2**	**0.28**	0.51	**2.72**	**99.1**	100	42.5	1.25
Bead-DAgger	98.4	99.1 [99.1, 99.1]	1.06 [1.05, 1.07]	2.47	5.47	**99.1**	100	42.4	1.25
Bead-DAgger→PPO	96.6	99.1 [99.0, 99.1]	1.77 [1.51, 2.14]	3.56	7.39	**99.1**	100	42.2	1.25
Bead-DAgger→PPO, prog. rew.	95.0	98.9 [98.8, 99.1]	2.14 [1.75, 2.31]	4.42	8.64	**99.1**	100	40.9	1.30
Bead-PPO (scratch)	69.5	89.8 [82.6, 94.8]	5.12 [4.25, 5.96]	10.64	19.27	96.4	98	36.8	1.36
Bead-PPO (scratch), prog. rew.	59.0	83.1 [76.8, 86.0]	6.49 [6.00, 7.73]	13.63	22.29	95.9	74	37.0	1.38
Bead, naive protocol*	77.0	93.9	4.38	9.26	16.30	98.9	100	42.5	1.23
Bicycle plant, native speed profile (Track A)									
Fixed PP (hand-set gains)	85.0	94.4	3.85	8.52	18.98	98.0	100	18.2	2.90
Fixed PP (tuned)	**99.0**	**99.2**	0.34	0.57	3.26	**99.1**	100	18.5	2.87
Adaptive PP (hand-set)	24.8	33.7	10.12	22.97	32.55	40.8	0	8.0	2.89
Adaptive PP (tuned)	98.9	**99.2**	0.42	0.69	3.83	**99.1**	100	18.5	2.87
RL–PP (joint L_d, g) [4]	94.2	96.6 [91.5, 99.1]	1.83 [1.27, 2.65]	3.89	11.35	97.0	96	18.1	2.86
RL–PP (L_d only)	97.7	99.0 [98.9, 99.1]	1.57 [1.21, 2.16]	3.23	10.14	**99.1**	100	18.7	2.86
RL–PP, naive protocol†	83.8	95.5	3.68	7.89	16.73	98.6	100	17.9	2.89
Bicycle plant, speed capped at $v_{\text{match}} = 1.256$ m/s (Track B)									
Fixed PP (tuned)	**99.0**	**99.2**	0.31	0.51	3.26	**99.1**	100	42.2	1.26
Adaptive PP (tuned)	98.9	**99.2**	0.35	**0.49**	3.74	**99.1**	100	42.3	1.26
RL–PP (joint L_d, g)	93.6	95.6 [89.8, 99.0]	2.40 [1.61, 4.10]	4.75	15.19	96.0	94	41.3	1.26

*Naive protocol (Section IV-B): actor regressed onto the expert, trained and selected on Hockenheim, Montreal and Yas Marina, three of the eight test circuits.
†Naive protocol (Section IV-B): trained on one hand-drawn contour; run on raw observations because its normalization statistics were not saved.
§The Fixed PP law of the car executed on the bead plant at v_{match} (Section III-G).

Fixed PP and RL–PP follow almost identically, and Bead-PPO from scratch cuts the corner.

B. Same Plant, Different Controller

The control law, not the vehicle, explains the gap. Run on the bead at v_{match}, the PP law reaches 0.28 cm, as accurate as on the car at the same speed (0.31 cm; the difference is not significant, Table VI) and well below the expert (1.07 cm), on all eight circuits (P4). Fig. 6 (left) shows that on the training circuits the error of the PP law depends on L_d in the same way on both plants, with the same optimum. The holonomic, damping-limited bead is therefore not harder to steer accurately than the car; the expert's rule is simply a worse controller than PP. Aiming the expert further ahead does not close the gap (Fig. 6, right). As defined in (4), the best target is the next waypoint ($k = 1$, 1.07 cm). With the nearest-point anchor, $k = 2$ is best (1.15 cm). Larger k makes the bead cut corners: from $k = 4$ on, some laps are no longer completed, and with the counter anchor the target locks once the counter stops advancing. Unlike PP, the expert does not turn the lateral offset of its target into a curvature command, and this, not its lookahead, limits its accuracy.

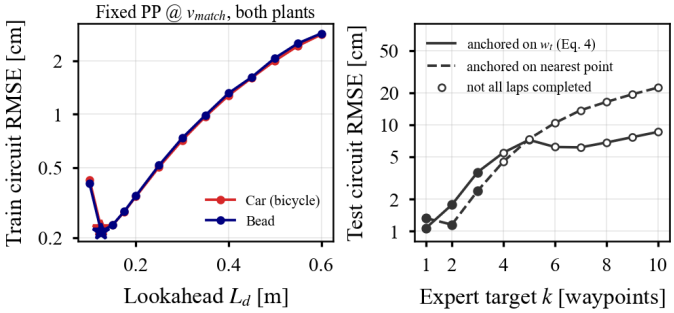


Fig. 6. Same plant, different controller. Left: training-circuit RMSE of the same Fixed PP law on the car and on the bead at v_{match} versus L_d (best gain at each L_d; only settings that complete every tuning lap; star: selected). Right: test-circuit RMSE of the expert (4) aiming k waypoints ahead, anchored on the waypoint counter or on the nearest reference point; open markers: not every test lap completed.

C. What the Naive Protocol Would Have Shown

The naive-protocol rows show how a weaker evaluation would have reversed the conclusion. On its own training circuits (Table VII) the naive bead reaches 2.45–4.05 cm RMSE, against 3.62–5.54 cm for Fixed PP with hand-set gains and 3.72–4.58 cm for naive RL–PP, so on each of the three circuits direct actuation appears the most precise of the three. Both parts of that picture are artifacts. With tuned gains, Fixed PP reaches 0.27–0.50 cm on the same three circuits, and the expert that the naive bead imitates reaches 0.94–1.36 cm. Over all eight test circuits, Fixed PP with hand-set gains reaches 3.85 cm and naive RL–PP, run without the normalization it was trained with, 3.68 cm. Untuned baselines alone are enough to reverse the ranking, and leakage inflates the bead's own figures: the naive bead averages 3.32 cm on its three training circuits but 5.02 cm on the five it has not seen.

D. What PPO Does to the Bead Policy

The PPO variants are genuinely optimized on their reward: actor, critic and action noise are all updated (verified by unit tests, including the exact transfer of the DAgger actor, its freezing during the critic warm-up, where the approximate KL divergence was exactly zero, and the behaviour-cloning term, which reduces to plain PPO when $\lambda = 0$). The training diagnostics are unremarkable: the final approximate KL per

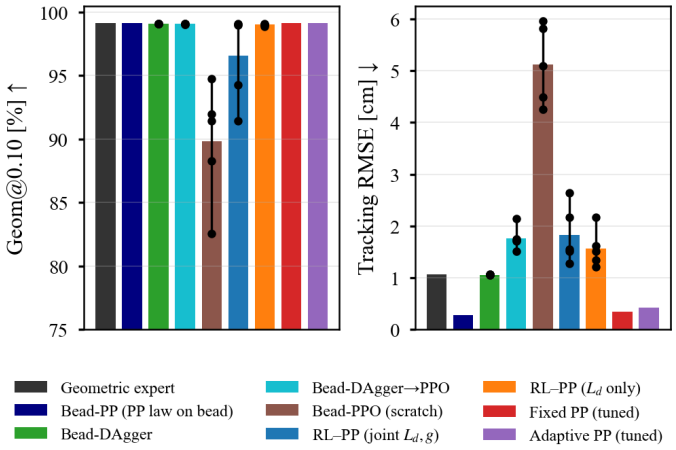


Fig. 7. Mean Geom@0.10 (left; larger is better) and RMSE (right; smaller is better) over the eight test circuits. Each dot is one training seed of a learned controller.

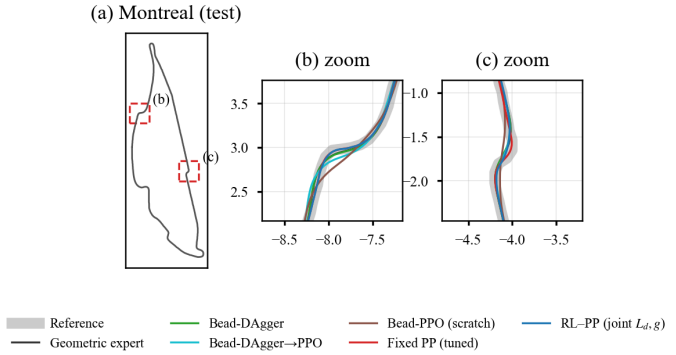


Fig. 8. Qualitative comparison on the held-out Montreal circuit (first episode, training seed 0). The zoom windows are centered, automatically, on the two reference points that Bead-DAgger→PPO misses by the largest distance, excluding the start/finish region.

update lies between 0.010 and 0.031, and the critic's explained variance reaches 0.86–0.98. Fig. 9 shows the validation error during training. PPO from scratch improves slowly and unevenly over 3×10^6 steps and remains far from the expert. Fine-tuning moves away from the DAgger policy despite the small initial noise, the lower learning rate and the behaviour-cloning term, and it does not come back within 2×10^6 steps. On the test circuits, fine-tuning raised the RMSE on all eight circuits for both rewards (P2 and Table VI).

On the test circuits the deterministic policies rank the same by base reward and by accuracy: the expert earns 715, Bead-DAgger 715, Bead-DAgger→PPO 694 and Bead-PPO 534. PPO, however, optimizes a stochastic policy, so we ran a validation-only diagnostic: each policy was executed on the four validation circuits with and without Gaussian action noise (three noise seeds, five training seeds), and its return was recorded under the reward it was trained on. With the base reward, deterministic Bead-DAgger earns 747 (RMSE 0.86 cm) and, with the initial fine-tuning noise, 749 (1.05 cm). After fine-tuning, the policy earns 729 with its own noise ($\sigma \approx 0.13$)

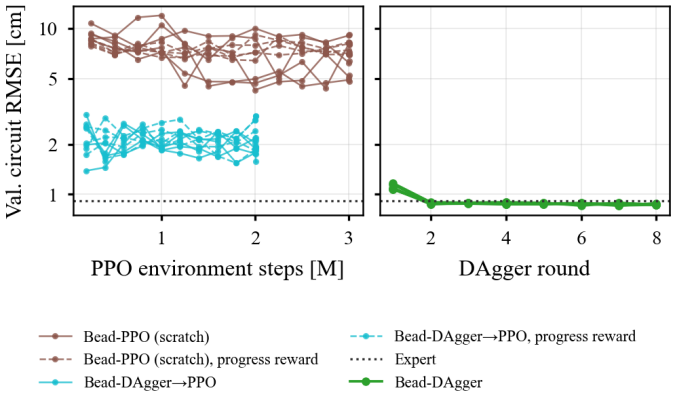


Fig. 9. Validation-circuit RMSE during training (five seeds each). Left: PPO from scratch and DAgger-initialized PPO, with the base reward (solid) and the progress reward (dashed); fine-tuning curves start after the critic warm-up. Right: the eight DAgger rounds of Algorithm 1. Dotted: geometric expert on the same validation circuits.

and 729 without (1.56 cm). With the progress reward, the corresponding returns are 838 and 829 before and 804 and 811 after fine-tuning (RMSE 1.84 cm deterministic). Thus, **within our budget, PPO fine-tuning lowered rather than raised the return of the stochastic policy it optimizes**, under either reward, so the loss of accuracy is not a mismatch between stochastic training and deterministic evaluation. The progress reward removes the incentive to drive slowly (Section III-E) but did not help: fine-tuned with it, the bead became faster and less accurate (2.14 cm on test). Which factor—the critic, the mixture of long circuits and short contours in training, the PPO step size, or the budget—drives the degradation was not isolated. We report this as a negative result: **with our training budget and settings, PPO did not improve the imitation policy**, and the expert remains the accuracy ceiling of every learned bead controller.

E. The Precision–Pace Trade-off

A bead lap lasts roughly 2.3 times as long as a lap of the PP car at its own speed (Fig. 10). This gap comes from the dynamics: according to (2), friction holds the bead near 1.3 m/s per axis, while the car follows a speed plan that reaches 5 m/s on straights. Accuracy cannot be compared fairly at different speeds; Track B addresses this. When the car's target speed is limited to v_{match}, its lap time becomes close to the bead's (42.2 versus 42.5 s, Table IV) and its error drops slightly: 0.31 cm for re-tuned Fixed PP and 0.35 cm for Adaptive PP, while retrained RL–PP reaches 2.40 cm. **At matched speed, tuned PP tracks the extracted references more closely than every learned bead controller and than the expert** (P1, all eight circuits), and the same law on the bead does as well (Section V-B). At native speed the car's accuracy is partly an artifact of the kinematic model, which lets it corner without slip (Section III-F).

F. Generalization to Held-Out Contours

Table V reports the 100 synthetic test contours. The ranking of the circuits carries over: Bead-PP and tuned Fixed PP are

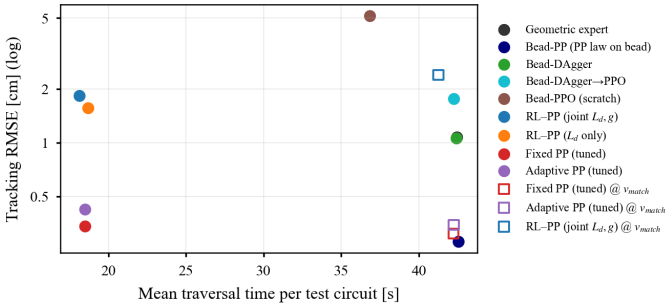


Fig. 10. Precision versus pace on the test circuits. Filled circles: native speed (Track A). Open squares: PP controllers with the reference speed capped at v_{match} (Track B).

TABLE V
RESULTS ON HELD-OUT SYNTHETIC TEST CONTOURS (MEAN ± STANDARD DEVIATION ACROSS CONTOURS; BEST PER SET IN BOLD)

Set / Controller	Geom@0.10 [%]	RMSE [cm]	Ordered [%]	Time [s]
Closed shapes (50)				
Geometric expert (4)	**99.1** ± 0.3	1.12	**99.1**	24.4
Bead-PP (PP law on bead)§	**99.1** ± 0.4	0.69	**99.1**	24.8
Bead-DAgger	**99.1** ± 0.4	1.17	**99.1**	24.4
Bead-DAgger→PPO	**99.1** ± 0.4	1.56	**99.1**	24.4
Bead-PPO (scratch)	98.2 ± 1.1	2.94	**99.1**	22.0
Fixed PP (tuned)	**99.1** ± 0.3	**0.63**	**99.1**	10.8
RL–PP (joint L_d, g) [4]	98.8 ± 0.5	1.46	**99.1**	10.8
Bead, naive protocol*	96.2 ± 2.8	3.90	99.0	25.1
RL–PP, naive protocol†	97.8 ± 1.4	2.35	99.0	10.7
Open curves (50)				
Geometric expert (4)	98.6 ± 0.1	1.06	**99.1**	15.5
Bead-PP (PP law on bead)§	98.6 ± 0.1	0.71	**99.1**	15.7
Bead-DAgger	98.6 ± 0.1	1.09	**99.1**	15.5
Bead-DAgger→PPO	98.6 ± 0.1	1.46	**99.1**	15.4
Bead-PPO (scratch)	98.4 ± 0.7	2.42	**99.1**	13.8
Fixed PP (tuned)	**98.7** ± 0.1	**0.69**	**99.1**	8.5
RL–PP (joint L_d, g) [4]	**98.7** ± 0.1	1.19	**99.1**	8.5
Bead, naive protocol*	98.4 ± 0.6	2.58	**99.1**	15.9
RL–PP, naive protocol†	98.5 ± 0.4	1.65	**99.1**	8.5

the most accurate, Bead-DAgger matches the expert, and the PPO-trained beads and naive-protocol checkpoints are less accurate. Coverage, however, saturates: the expert, Bead-PP, Bead-DAgger, Bead-DAgger→PPO and tuned Fixed PP all reach the largest Geom@0.10 that the lap rule allows (99.1% on closed shapes, 98.6–98.7% on open curves), so among them only RMSE separates the controllers.

G. Interpretability of the Learned PP Schedule

Fig. 11 shows how the joint RL–PP agent sets its parameters on Montreal. The lookahead ranges over 0.05–0.86 m and falls as the upcoming curvature rises ($r = -0.68$), matching the trend described in [4]: the agent looks closer before corners and farther on straights. With the lower bound at 0.05 m the lookahead is no longer pinned to the bound. The steering gain varies between 0.87 and 1.26 (standard deviation 0.09) and increases in corners ($r = +0.70$), as the teacher prescribes. The schedule also explains why RL–PP is less accurate than tuned constant PP. Over all test circuits and five seeds its median lookahead is 0.44 m, and 57% of its steps lie inside the teacher's band $[0.26, 0.85]$ m, far above the 0.125 m that tuning selects for Fixed PP. Its reward pulls it towards a teacher

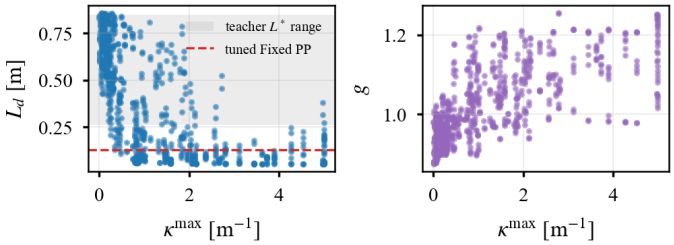


Fig. 11. Learned joint RL–PP schedule on the held-out Montreal circuit (training seed 0): L_d (left) and g (right) versus previewed curvature $\kappa_{\max}$. Shaded: range of the teacher's lookahead L^*; dashed: lookahead selected for tuned Fixed PP.

that favours long lookaheads on straights. We did not redesign the teacher, which would have meant tuning the method of [4] beyond adapting it.

H. Ablation: Joint versus Lookahead-Only Tuning

The L_d-only agent is trained on its own (Section III-F). Dropping the gain output does not hurt: the L_d-only agent is slightly more accurate than the joint one (1.57 versus 1.83 cm; the bootstrap interval includes zero, Table VI), and neither approaches tuned Fixed PP (P3). In our small arena and with our budget, learned PP parameters offered no advantage over good constant ones; this says nothing against the hardware results of [4].

I. Statistical Evidence

Table VI lists the comparisons. The bootstrap intervals of all four primary comparisons exclude zero, and in each of them the same controller is better on all eight circuits. The exact Wilcoxon test therefore gives its smallest attainable value, $p = 0.0078$, which remains below 0.05 after Holm correction over the four tests ($p = 0.031$). Direct actuation is less accurate than tuned PP at equal speed (P1), PPO fine-tuning makes the imitation policy less accurate (P2), learned PP parameters are less accurate than tuned constant ones (P3), and the PP law is more accurate than the expert on the same plant (P4). Among the exploratory comparisons, Bead-DAgger and the expert cannot be told apart, nor can the PP law on the bead and on the car, while the progress reward made both PPO procedures less accurate. Two cautions apply. With eight circuits, a p of 0.0078 only says that all circuits agree; the size of an effect is given by its interval. And the intervals describe variation over these circuits and five training seeds, not over other simulators, track scales or vehicles.

J. Threats to Validity

- **Prior exposure to test circuits.** No test circuit is used for training, checkpoint selection, or tuning, but Hockenheim, Montreal and Yas Marina were known to the authors when the reward and the network were designed, before the split was fixed.
- **Different plants.** The bead is holonomic and damping-limited, the car non-holonomic. Bead-PP and the

TABLE VI

PAIRED RMSE COMPARISONS ON THE EIGHT TEST CIRCUITS (DIFFERENCE A−B IN CM, NEGATIVE FAVOURS A; 95% BOOTSTRAP INTERVAL OVER CIRCUITS AND TRAINING SEEDS; EXACT WILCOXON SIGNED-RANK TEST)

Comparison (A vs B)	Mean diff.	95% bootstrap CI	A better on	Wilcoxon p	Holm p
Primary (Holm over these four)					
DAgger vs Fixed PP @v_{match}	+0.75	[+0.62, +0.87]	0/8	0.0078	0.031
DAgger→PPO vs DAgger	+0.71	[+0.54, +0.94]	0/8	0.0078	0.031
RL–PP joint vs Fixed PP	+1.49	[+1.06, +2.02]	0/8	0.0078	0.031
Bead-PP vs Expert	−0.79	[−0.92, −0.64]	8/8	0.0078	0.031
Exploratory (no correction)					
DAgger vs Expert	−0.01	[−0.06, +0.04]	5/8	0.8438	–
DAgger→PPO vs Expert	+0.69	[+0.53, +0.93]	0/8	0.0078	–
PPO (scratch) vs DAgger→PPO	+3.36	[+2.35, +5.01]	0/8	0.0078	–
DAgger→PPO (prog.) vs DAgger→PPO	+0.37	[+0.08, +0.64]	0/8	0.0078	–
PPO (scratch, prog.) vs PPO (scratch)	+1.37	[+0.18, +2.32]	0/8	0.0078	–
Bead-PP vs Fixed PP @v_{match}	−0.03	[−0.08, +0.03]	5/8	0.5469	–
Fixed PP @v_{match} vs Expert	−0.76	[−0.86, −0.66]	8/8	0.0078	–
RL–PP joint vs RL–PP L_d	+0.26	[−0.32, +0.87]	1/8	0.0234	–
Adaptive PP vs Fixed PP	+0.08	[+0.06, +0.11]	0/8	0.0078	–

TABLE VII

GEOM@0.10 [%] / RMSE [CM] ON HOCKENHEIM, MONTREAL AND YAS MARINA, THE TRAINING CIRCUITS OF THE NAIVE-PROTOCOL BEAD (BEST PER COLUMN IN BOLD)

	Hockenheim		Montreal		Yas Marina	
Controller	G@0.10	RMSE	G@0.10	RMSE	G@0.10	RMSE
Geometric expert (4)	**99.1**	0.94	**99.3**	1.24	**99.1**	1.36
Bead-DAgger	**99.1**	0.87	**99.3**	1.35	**99.1**	1.40
Bead, naive protocol*	**99.1**	2.45	96.1	3.45	95.6	4.05
Fixed PP (hand-set gains)	95.7	3.62	90.8	5.54	89.8	5.17
Fixed PP (tuned)	**99.1**	**0.27**	**99.3**	**0.41**	**99.1**	**0.50**
RL–PP (joint L_d, g) [4]	98.7	1.64	99.0	1.60	95.5	2.40
RL–PP, naive protocol†	96.1	3.72	94.2	4.40	91.9	4.58

matched-speed track control for this, but they cover one control law and one speed.

- **PPO budget and settings.** PPO ran for 3×10^6 steps from scratch and 2×10^6 steps as fine-tuning, with one hyper-parameter configuration each; the fine-tuning remedies (initial noise, learning rate, behaviour-cloning weight) were set in advance, not tuned, and their individual effects were not isolated. The results show that PPO did not improve the policy under these settings, not that it cannot.
- **Training seeds.** Five seeds per learned controller describe seed variation only coarsely.
- **Reward design.** The base reward pays for time near the path; the progress variant removes that incentive, but neither reward was tuned, and other designs might let PPO refine the policy.
- **RL–PP adaptation.** Our RL–PP differs from [4] in scale, ranges, teacher, reward weights and budget (Table III); its teacher favours lookaheads longer than the tuned optimum, so its results characterize our adaptation, not the original method.
- **Reference-extraction error.** The extraction error (up to 6.53 cm, about 1 cm on average) is of the same order as the tracking errors; differences below about 1 cm concern the extracted path, not the real track.
- **Track scale and vehicle model.** The circuits are shrunk 4.3–9.8 times, and the kinematic car ignores tyre slip; at native speed it demands lateral accelerations no tyre could provide, which flatters the native-speed car results.
- **Evaluation scope.** Only 50 of 150 synthetic test contours per set were evaluated, and these share their generator with the training contours. A preliminary run of Bead-PP and the expert sweep had touched the test circuits before the final settings were fixed; no setting was chosen from it.
- **Simulation only.** All results are in simulation at arena scale, with no physical experiments.

VI. CONCLUSION

This work tested, under shared paths, shared measures and a shared lap rule, whether a network that commands actuation directly can out-track a PP controller with learned or tuned parameters. The direct controller was a point-mass bead fed by an image-to-path front end and trained with DAgger, with PPO, or with both in sequence; the comparison set consisted of our adaptation of RL-tuned PP [4], two classical PP controllers, and the PP law run on the bead.

With test circuits held out and all baselines tuned on training data, the answer is no. The DAgger-trained bead reproduces its geometric expert, so the direct-actuation accuracy is the expert's, not a product of learning. With our training budgets and settings, PPO did not improve on that expert, from scratch or as fine-tuning, with small initial noise, a lower learning rate and a behaviour-cloning term, and with a reward that pays for accuracy rather than for time near the path. A Fixed PP with tuned gains, whose optimum lies inside the searched range, tracks more closely at more than twice the pace and at matched speed. Run on the bead, the same PP law is as accurate as on the car, so the gap comes from the control law, not from the vehicle. Our adaptation of RL–PP reproduces the qualitative schedule of [4] but follows a teacher with long lookaheads and is less accurate than tuned Fixed PP.

These differences agree in sign on all eight test circuits; their sizes are given by the bootstrap intervals, and all are measured against extracted references whose own error is about 1 cm. A naive protocol, scoring the bead on its training circuits against PP with hand-set gains, would have reported the opposite conclusion. The practical lesson is that a well-tuned geometric law is a strong baseline, which learned controllers should be compared against before claims of better accuracy are made.

Next steps follow from the limitations: (a) isolating why PPO degrades the imitation policy, for example by tuning the behaviour-cloning weight or using constrained policy updates, and training much longer; (b) a learned residual on top of the PP law instead of replacing it; (c) making path extraction more accurate than the tracking errors being measured; and (d) testing on a dynamic vehicle model with tyre slip and on a real F1TENTH car.

Acknowledgment

We are grateful to the F1TENTH community for publishing its track data, and to Elgouhary and El-Wakeel [4], whose detailed description of their method allowed us to adapt it to our simulator.

References

[1] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1TENTH: An open-source evaluation environment for continuous control and reinforcement learning," in *Proc. NeurIPS 2019 Competition and Demonstration Track*, PMLR vol. 123, 2020, pp. 77–89.

[2] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Carnegie Mellon Univ., Robotics Inst., Tech. Rep. CMU-RI-TR-92-01, 1992.

[3] J. M. Snider, "Automatic steering methods for autonomous automobile path tracking," Carnegie Mellon Univ., Robotics Inst., Tech. Rep. CMU-RI-TR-09-08, 2009.

[4] M. Elgouhary and A. S. El-Wakeel, "Learning to tune pure pursuit in autonomous racing: Joint lookahead and steering-gain control with PPO," arXiv:2602.18386, 2026.

[5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[6] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. AISTATS*, 2011, pp. 627–635.

[7] G. M. Hoffmann, C. J. Tomlin, M. Montemerlo, and S. Thrun, "Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and racing," in *Proc. American Control Conf.*, 2007, pp. 2296–2301.

[8] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. New York, NY, USA: Springer, 2012.

[9] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli, "Kinematic and dynamic vehicle models for autonomous driving control design," in *Proc. IEEE Intelligent Vehicles Symp.*, 2015, pp. 1094–1099.

[10] A. Heilmeier, A. Wischnewski, L. Hermansdorfer, J. Betz, M. Lienkamp, and B. Lohmann, "Minimum curvature trajectory planning and control for an autonomous race car," *Vehicle System Dynamics*, vol. 58, no. 10, pp. 1497–1527, 2020.

[11] J. Betz *et al.*, "Autonomous vehicles on the edge: A survey on autonomous vehicle racing," *IEEE Open J. Intell. Transp. Syst.*, vol. 3, pp. 458–488, 2022.

[12] B. R. Kiran *et al.*, "Deep reinforcement learning for autonomous driving: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 6, pp. 4909–4926, 2022.

[13] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *Int. J. Robotics Research*, vol. 32, no. 11, pp. 1238–1274, 2013.

[14] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. ICLR*, 2016.

[15] M. Andrychowicz *et al.*, "What matters in on-policy reinforcement learning? A large-scale empirical study," in *Proc. ICLR*, 2021.

[16] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. ICML*, 2018, pp. 1861–1870.

[17] B. D. Evans, H. W. Jordaan, and H. A. Engelbrecht, "Comparing deep reinforcement learning architectures for autonomous racing," *Machine Learning with Applications*, vol. 14, p. 100496, 2023.

[18] D. A. Pomerleau, "ALVINN: An autonomous land vehicle in a neural network," in *Advances in Neural Information Processing Systems 1*, 1989, pp. 305–313.

[19] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Trans. Syst., Man, Cybern.*, vol. 9, no. 1, pp. 62–66, 1979.

[20] T. Y. Zhang and C. Y. Suen, "A fast parallel algorithm for thinning digital patterns," *Commun. ACM*, vol. 27, no. 3, pp. 236–239, 1984.

[21] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.

[22] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," arXiv:2407.17032, 2024.